

Bugs Managed

Azure Deployment Guide

Step-by-step instructions to deploy the Bugs Managed platform to Azure.

What You're Deploying

- **Bugs Managed API** - ASP.NET Core backend (Azure App Service)
- **Bugs Managed Dashboard** - React admin portal (Azure Static Web App)
- **Azure SQL Database** - SQL Server database (managed)

Once deployed, all our apps (Financials Managed, HDD Managed, Facilities Managed, etc.) will send bug reports and feature requests to this central platform.

Prerequisites

Before you start, make sure you have:

1. **Azure account** with an active subscription
2. **Azure CLI** installed - <https://learn.microsoft.com/en-us/cli/azure/install-azure-cli>
3. **Git** installed
4. **Node.js 20+** installed - <https://nodejs.org>
5. **.NET 10 SDK** installed - <https://dotnet.microsoft.com/download>
6. Access to the GitHub repo

Step 1: Clone the Repo

```
git clone https://github.com/larry0467/bugs-managed-sqlserver-aspnet-react.git
cd bugs-managed-sqlserver-aspnet-react
```

Step 2: Log in to Azure

```
az login
```

This opens a browser window. Sign in with the Azure account.

Step 3: Create Azure Resources

Run these commands one at a time. Replace the password and names if needed.

3a. Create a Resource Group

```
az group create --name bugs-managed-rg --location eastus
```

3b. Create Azure SQL Server

```
az sql server create \  
--name bugs-managed-sql \  
--resource-group bugs-managed-rg \  
--location eastus \  
--admin-user bugsadmin \  
--admin-password "BugsManaged2026!"
```

IMPORTANT: Save this password. You will need it in Step 5.

3c. Create the Database

```
az sql db create \  
--resource-group bugs-managed-rg \  
--server bugs-managed-sql \  
--name bugs_managed \  
--service-objective S0
```

3d. Allow Azure Services to Access the Database

```
az sql server firewall-rule create \  
--resource-group bugs-managed-rg \  
--server bugs-managed-sql \  
--name AllowAzureServices \  
--start-ip-address 0.0.0.0 \  
--end-ip-address 0.0.0.0
```

3e. Create App Service Plan

```
az appservice plan create \  
--name bugs-managed-plan \  
--resource-group bugs-managed-rg \  
--sku B1 \  
--is-linux
```

3f. Create the API App Service

```
az webapp create \  
--resource-group bugs-managed-rg \  
--plan bugs-managed-plan \  
--name bugs-managed-api
```

```
--runtime "DOTNETCORE:10.0"
```

NOTE: The name bugs-managed-api must be globally unique. If taken, use something like bugs-managed-api-yourcompany.

Step 4: Create the Static Web App for the Dashboard

```
az staticwebapp create \  
--name bugs-managed-admin \  
--resource-group bugs-managed-rg \  
--location eastus
```

Step 5: Configure the API

This sets the database connection, JWT secret, and CORS origins.

```
az webapp config appsettings set \  
--resource-group bugs-managed-rg \  
--name bugs-managed-api \  
--settings \  
"ConnectionStrings__DefaultConnection=Server=tcp:bugs-managed-sql.database.windows.net,1433;Database=bugs_managed;User ID=bugsadmin;Password=BugsManaged2026!;Encrypt=True;TrustServerCertificate=False;" \  
"BugsManaged__Jwt__Secret=BugsManaged2026SecretKeyMustBeAtLeast256BitsLong!" \  
"BugsManaged__Jwt__ExpirationMs=86400000" \  
"BugsManaged__VideoStoragePath=/home/videos" \  
"BugsManaged__Cors__AllowedOrigins=https://bugs-managed-admin.azurestaticapps.net,https://financials.protocall.co,https://hdd.protocall.co"
```

IMPORTANT: Update the CORS origins to include all your app domains. Comma-separated, no spaces.

Step 6: Deploy the API

Option A: Deploy from command line (quickest)

```
cd BugsManaged.Api  
dotnet publish -c Release -o ./publish  
cd publish  
zip -r ../deploy.zip .  
cd ..  
  
az webapp deployment source config-zip \  
--resource-group bugs-managed-rg \  
--name bugs-managed-api \  
--src deploy.zip
```

Option B: Set up GitHub Actions (auto-deploy on every push)

1. Get the publish profile:

```
az webapp deployment list-publishing-profiles \  
--name bugs-managed-api \  
--resource-group bugs-managed-rg \  
--xml > publish-profile.xml
```

2. Go to the GitHub repo > Settings > Secrets and variables > Actions
3. Create a new secret: Name: **AZURE_WEBAPP_PUBLISH_PROFILE**, Value: paste the entire contents of publish-profile.xml
4. Delete the local file: **rm publish-profile.xml**

Step 7: Deploy the Admin Dashboard

7a. Build the dashboard

```
cd bugs-managed-admin
npm install
npm run build
```

7b. Get the Static Web App deployment token

```
az staticwebapp secrets list \
--name bugs-managed-admin \
--resource-group bugs-managed-rg \
--query "properties.apiKey" -o tsv
```

7c. Install the SWA CLI and deploy

```
npm install -g @azure/static-web-apps-cli

swa deploy ./dist \
--deployment-token <paste-token-from-step-7b> \
--env production
```

7d. (Optional) Set up GitHub Actions for auto-deploy

1. Go to GitHub repo > Settings > Secrets > Actions
2. Create secret: Name: **AZURE_STATIC_WEB_APPS_TOKEN**, Value: the token from step 7b
3. Now every push to master auto-deploys both the API and dashboard.

Step 8: Verify the Deployment

1. **API health check:** Open <https://bugs-managed-api.azurewebsites.net/api/auth/me> in a browser. You should see: `{"error":"Missing or invalid Authorization header"}` - That means the API is running!
2. **Dashboard:** Open <https://bugs-managed-admin.azurestaticapps.net> - You should see the login page.
3. **Register your admin account:** Click Register, enter your name, email, password, organization name.
4. **Create projects for each app:** Go to Applications > New Project. Create: Financials Managed, HDD Managed, Facilities Managed, etc. Copy each API key.

Step 9: Connect Your Apps

For each app (Financials Managed, HDD Managed, etc.):

9a. Copy the widget files into the app

Copy these two files from the repo into the app's src/components/ folder:

- bugs-managed-widget/src/BugsManagedWidget.tsx
- bugs-managed-widget/src/types.ts (rename to BugsManagedTypes.ts)

Update the import in BugsManagedWidget.tsx:

```
// Change this:
import type { BugsManagedConfig } from './types';
// To this:
import type { BugsManagedConfig } from './BugsManagedTypes';
```

9b. Add the widget to the app

In the app's main layout component (e.g., App.tsx):

```
import BugsManagedWidget from './components/BugsManagedWidget';

// Inside the JSX, near the closing tags:
<BugsManagedWidget
  apiKey="bm_PASTE_API_KEY_FROM_STEP_8"
  apiUrl="https://bugs-managed-api.azurewebsites.net/api"
  userEmail={currentUser.email}
  userName={currentUser.name}
  tenantName={currentTenant.name}
  tenantId={currentTenant.id}
  databaseName={currentTenant.dbName}
  appVersion="1.0.0"
  environment="PRODUCTION"
  theme="dark"
  position="bottom-right"
/>
```

Replace currentUser and currentTenant with however your app tracks the logged-in user and selected company/tenant.

9c. Deploy the app

Deploy your app as usual. The widget will now send bug reports to the central Bugs Managed dashboard.

Step 10: Add Team Members

1. Log in to the Bugs Managed dashboard
2. Go to **Team** in the sidebar
3. Click **Add Team Member**
4. Enter their name, email, temporary password, and role:
 - **Platform Admin** - full access, can manage team and all projects
 - **Project Admin** - can manage tickets and projects
 - **Viewer** - read-only
5. Share their login credentials. They log in at the dashboard URL.

Step 11: Configure Slack (Optional)

1. Go to **Settings** in the dashboard
2. Select the project (e.g., "Financials Managed")
3. Paste your **Slack Incoming Webhook URL**
4. Enter the **Slack channel** name (e.g., #bugs-financials)
5. Click **Save**

Now all ticket chat messages will sync to Slack.

For inbound messages from Slack:

1. Create a Slack App at <https://api.slack.com/apps>
2. Add slash command: /bug-chat pointing to <https://bugs-managed-api.azurewebsites.net/api/slack/command>
3. Install the app to your workspace

Usage: /bug-chat 42 Looking into this now - posts a note to ticket #42.

Summary of URLs

Resource	URL
API	https://bugs-managed-api.azurewebsites.net
Dashboard	https://bugs-managed-admin.azurestaticapps.net
Widget API URL	https://bugs-managed-api.azurewebsites.net/api

Troubleshooting

"CORS error" in browser console:

Add the app's domain to the CORS setting in Step 5. No redeploy needed - app settings take effect immediately.

"Invalid API key" when widget submits:

Make sure the API key in the widget matches the one shown in Applications for that project.

Dashboard shows empty projects:

Make sure you're logged in with the same account that created the projects.

Database connection fails:

Verify the firewall rule in Step 3d was created. Check the connection string password matches.

Cost Estimate (Azure)

Resource	SKU	Monthly Cost
----------	-----	--------------

App Service (API)	B1	~\$13
Azure SQL	S0	~\$15
Static Web App (Dashboard)	Free	\$0
Total		~\$28/month

You can scale up later if needed. The S0 SQL tier handles hundreds of concurrent users.